```
#----imports and dataset preparation---------
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")  # to ignore warnings

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load dataset
data = load_breast_cancer()
X, y = data.data, data.target

# Standardize features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split into train-test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training samples: {X_train.shape[0]}, Test samples: {X_test.shape[0]}")
```

```
Training samples: 455, Test samples: 114
```

```
#-------------------------------------------------
#helper function to evaluvate the model
#-------------------------------------------------
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix, roc_curve, auc

def evaluate_model(model, X_test, y_test, name="Model"):
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print(f"{name} -> Accuracy: {acc:.4f}, F1 Score: {f1:.4f}")
    print(classification_report(y_test, y_pred))

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.title(f"Confusion Matrix - {name}")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

    # ROC Curve
    y_prob = model.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    plt.plot(fpr, tpr, label=f"{name} (AUC={auc(fpr,tpr):.2f})")
    plt.plot([0, 1], [0, 1], "k--")
    plt.legend()
    plt.title(f"ROC Curve - {name}")
    plt.show()

    return acc, f1



#-----------------------------------------------------------
#decision tree with grid search
#-----------------------------------------------------------
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

dt_params = {"criterion": ["gini", "entropy"], "max_depth": [3, 5, 7, None]}
dt = GridSearchCV(DecisionTreeClassifier(random_state=42), dt_params, cv=5)
dt.fit(X_train, y_train)

print("Best DT Params:", dt.best_params_)
```

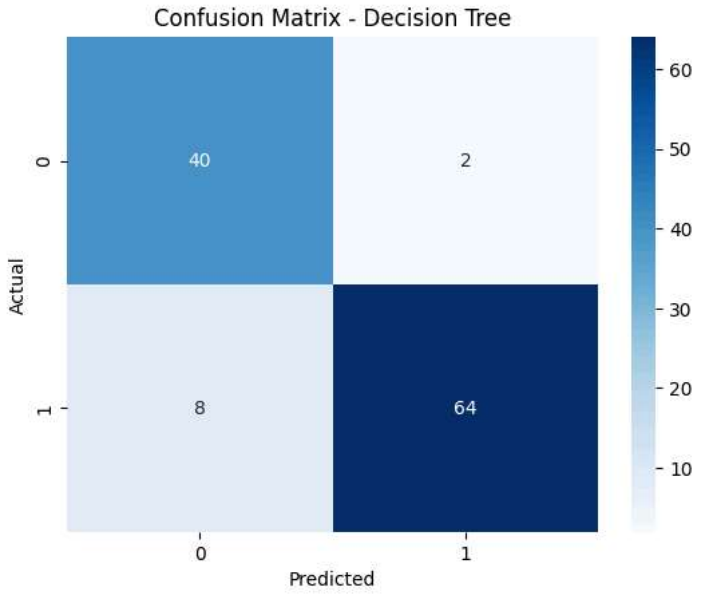
```
dt_acc, dt_f1 = evaluate_model(dt.best_estimator_, X_test, y_test, "Decision Tree")

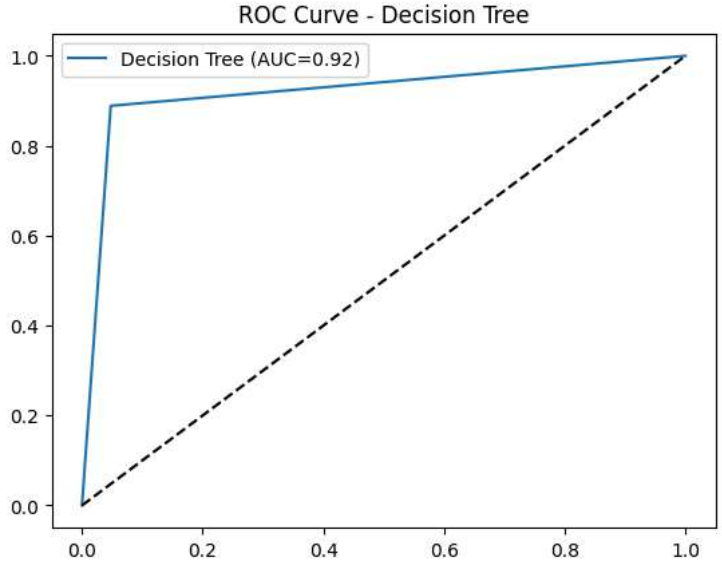
dt_results = pd.DataFrame(dt.cv_results_)
dt_results.to_csv("DecisionTree_tuning.csv", index=False)
```

```
Best DT Params: {'criterion': 'entropy', 'max_depth': 7}
Decision Tree -> Accuracy: 0.9123, F1 Score: 0.9275
              precision    recall  f1-score   support

           0       0.83      0.95      0.89        42
           1       0.97      0.89      0.93        72

    accuracy                           0.91       114
   macro avg       0.90      0.92      0.91       114
weighted avg       0.92      0.91      0.91       114
```






```
#-----------------------------------------------------------
#AdaBoost
#-----------------------------------------------------------
from sklearn.ensemble import AdaBoostClassifier

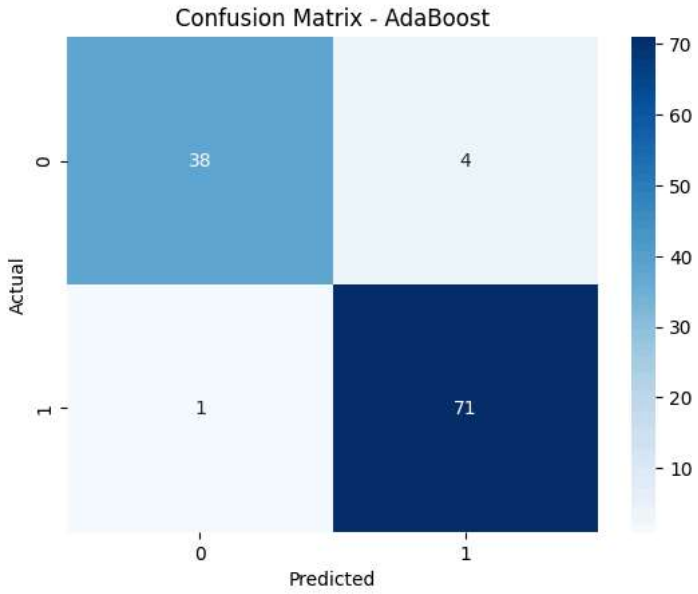
ada_params = {"n_estimators": [50, 100], "learning_rate": [0.01, 0.1, 1]}
ada = GridSearchCV(AdaBoostClassifier(random_state=42), ada_params, cv=5)
ada.fit(X_train, y_train)

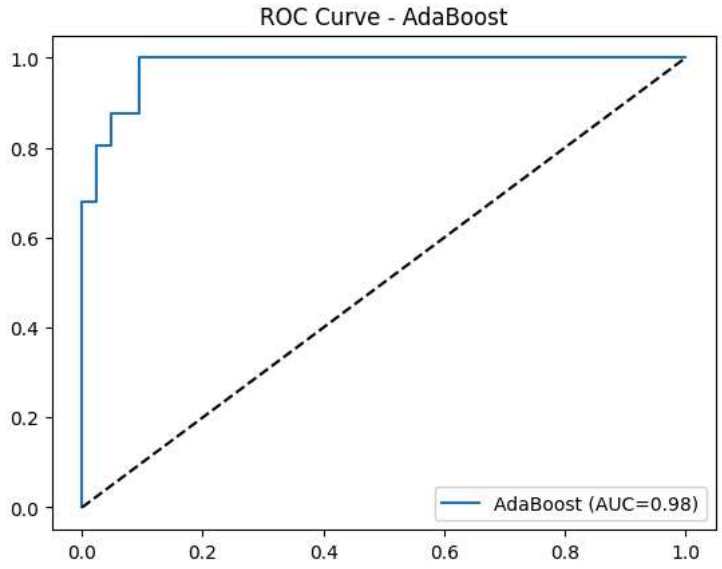
print("Best Ada Params:", ada.best_params_)
ada_acc, ada_f1 = evaluate_model(ada.best_estimator_, X_test, y_test, "AdaBoost")
```

```
Best Ada Params: {'learning_rate': 1, 'n_estimators': 100}
AdaBoost -> Accuracy: 0.9561, F1 Score: 0.9660
              precision    recall  f1-score   support

           0       0.97      0.90      0.94        42
           1       0.95      0.99      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```






```
#-----------------------------------------------------------
#Gradient Boosting
#-----------------------------------------------------------
from sklearn.ensemble import GradientBoostingClassifier

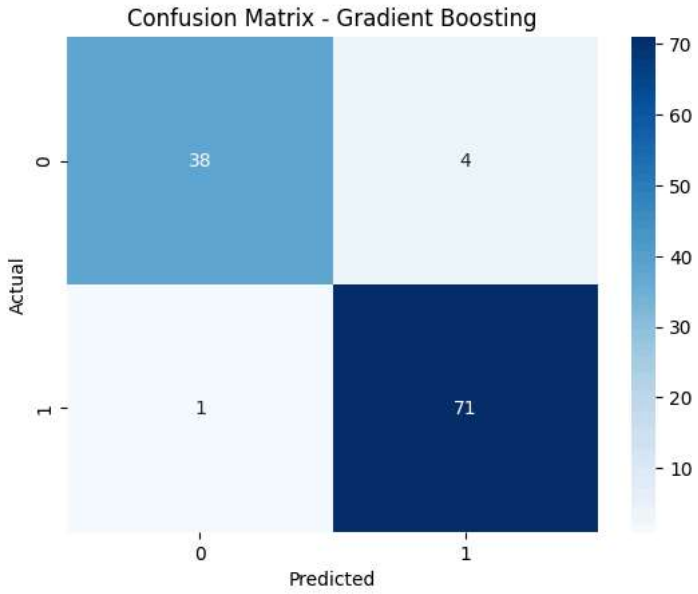
gb_params = {"n_estimators": [50, 100], "learning_rate": [0.01, 0.1, 0.5], "max_depth": [3, 5]}
gb = GridSearchCV(GradientBoostingClassifier(random_state=42), gb_params, cv=5)
gb.fit(X_train, y_train)

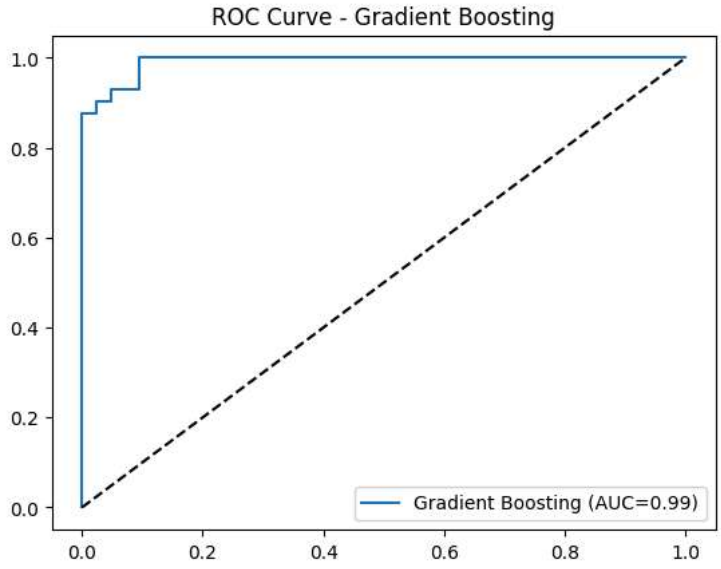
print("Best GB Params:", gb.best_params_)
gb_acc, gb_f1 = evaluate_model(gb.best_estimator_, X_test, y_test, "Gradient Boosting")
```

```
Best GB Params: {'learning_rate': 0.5, 'max_depth': 3, 'n_estimators': 50}
Gradient Boosting -> Accuracy: 0.9561, F1 Score: 0.9660
              precision    recall  f1-score   support

           0       0.97      0.90      0.94        42
           1       0.95      0.99      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```






```
#-----------------------------------------------------------
#Xg Boost
#-----------------------------------------------------------
import xgboost as xgb

xgb_params = {
    "n_estimators": [50, 100],
    "learning_rate": [0.01, 0.1],
    "max_depth": [3, 5],
    "gamma": [0, 1]
}

xgb_clf = GridSearchCV(
    xgb.XGBClassifier(
        objective="binary:logistic",  # classification task
        eval_metric="logloss",        # avoids use_label_encoder warning
        use_label_encoder=False,
        random_state=42
```

```
    ),
    xgb_params,
    cv=5
)

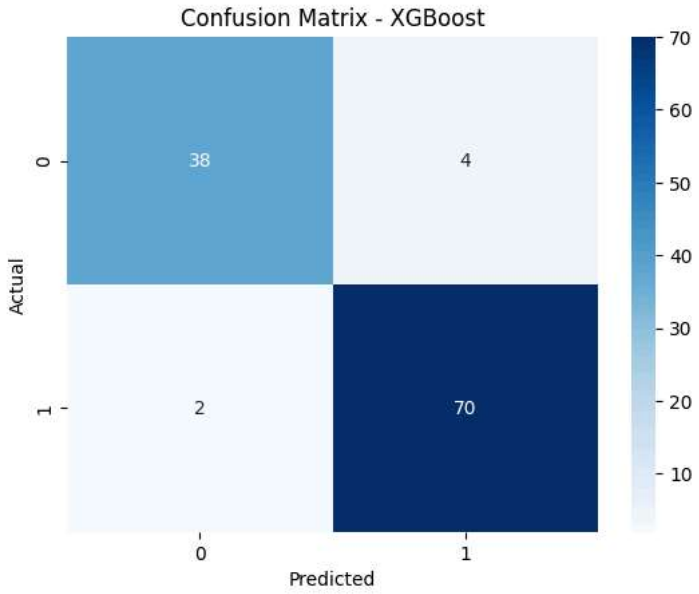
xgb_clf.fit(X_train, y_train)
print("Best XGB Params:", xgb_clf.best_params_)

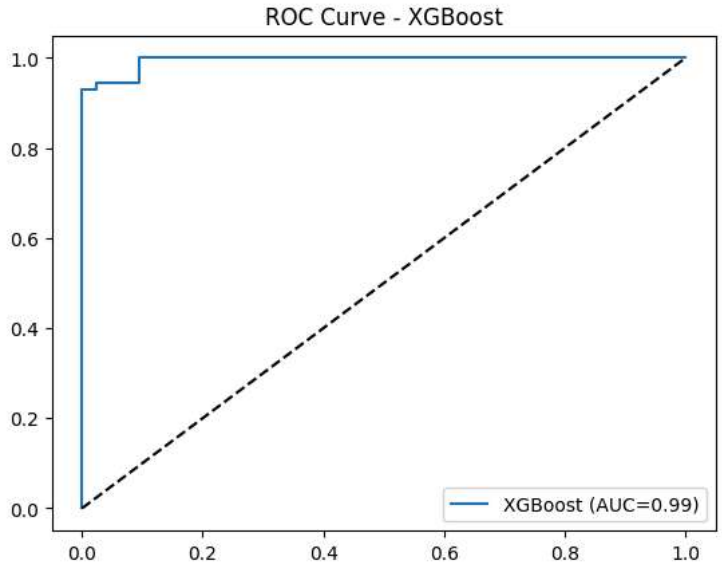
xgb_acc, xgb_f1 = evaluate_model(xgb_clf.best_estimator_, X_test, y_test, "XGBoost")
```

```
Best XGB Params: {'gamma': 0, 'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 100}
XGBoost -> Accuracy: 0.9474, F1 Score: 0.9589
              precision    recall  f1-score   support

           0       0.95      0.90      0.93        42
           1       0.95      0.97      0.96        72

    accuracy                           0.95       114
   macro avg       0.95      0.94      0.94       114
weighted avg       0.95      0.95      0.95       114
```






```
#-------------------------------------------------------
#Random forest
#-------------------------------------------------------
from sklearn.ensemble import RandomForestClassifier

rf_params = {
    "n_estimators": [50, 100],
```

```
    "max_depth": [3, 5, None],
    "criterion": ["gini", "entropy"]
}

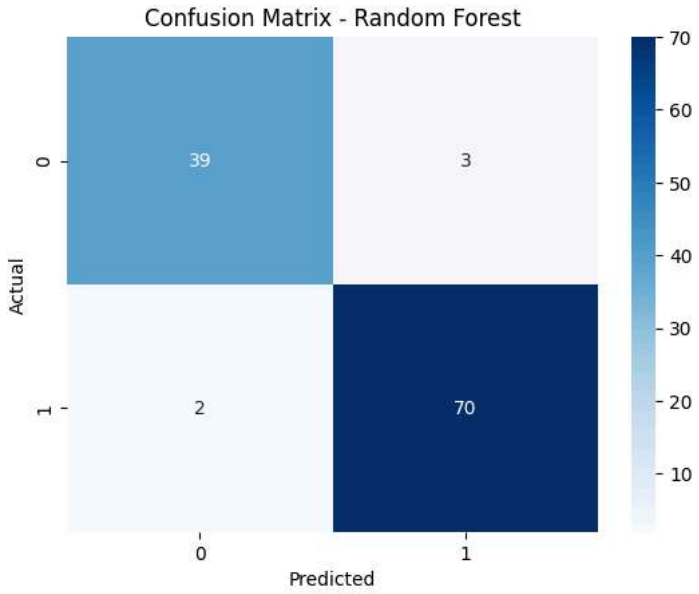
rf = GridSearchCV(RandomForestClassifier(random_state=42), rf_params, cv=5)
rf.fit(X_train, y_train)

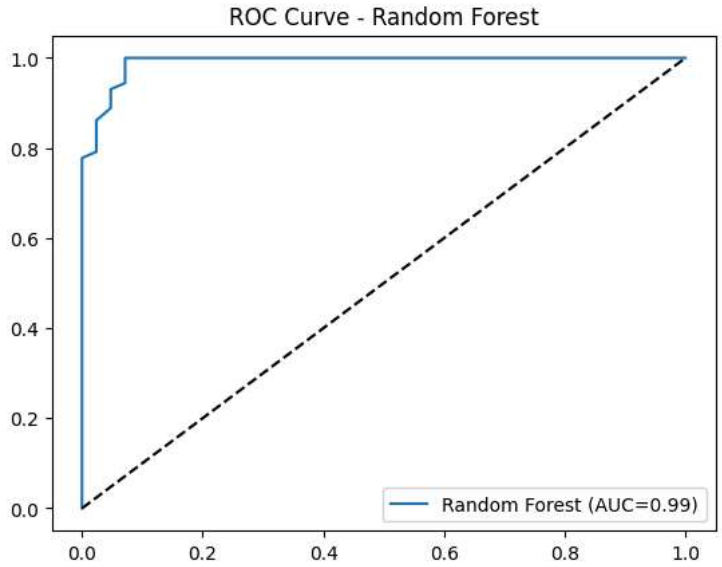
print("Best RF Params:", rf.best_params_)
rf_acc, rf_f1 = evaluate_model(rf.best_estimator_, X_test, y_test, "Random Forest")
```

```
Best RF Params: {'criterion': 'entropy', 'max_depth': None, 'n_estimators': 50}
Random Forest -> Accuracy: 0.9561, F1 Score: 0.9655
              precision    recall  f1-score   support

           0       0.95      0.93      0.94        42
           1       0.96      0.97      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```






```
#-------------------------------------------------------
#Stacked Ensemble (SVM + Naive Bayes + Decision Tree)
#-------------------------------------------------------
from sklearn.ensemble import StackingClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
```

```
estimators = [
    ("svm", SVC(probability=True, kernel="linear")),
    ("nb", GaussianNB()),
    ("dt", DecisionTreeClassifier(max_depth=3))
]

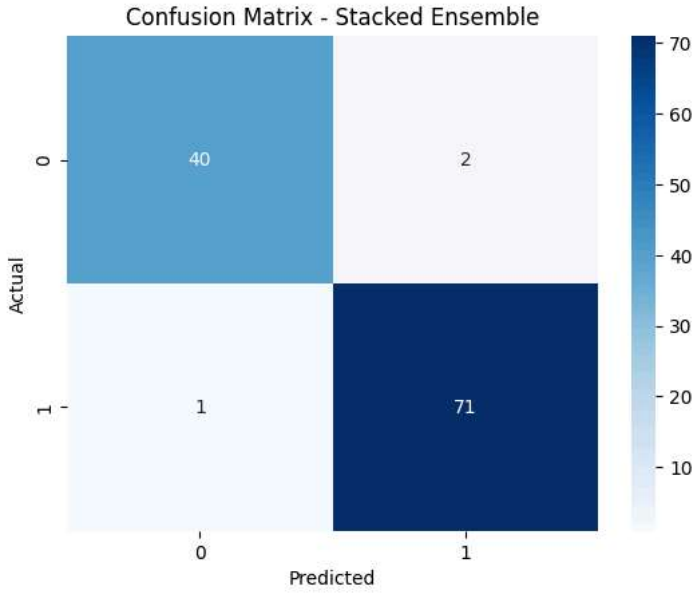
stack = StackingClassifier(estimators=estimators, final_estimator=LogisticRegression())
stack.fit(X_train, y_train)

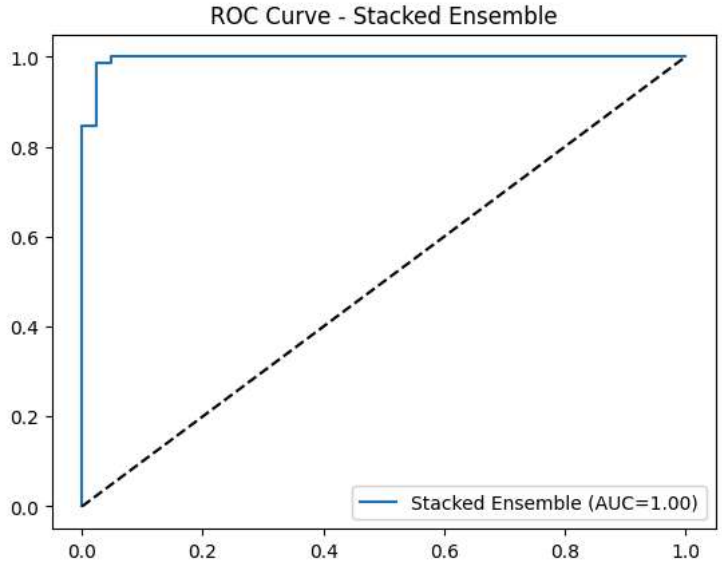
stack_acc, stack_f1 = evaluate_model(stack, X_test, y_test, "Stacked Ensemble")
```

```
Stacked Ensemble -> Accuracy: 0.9737, F1 Score: 0.9793
              precision    recall  f1-score   support

           0       0.98      0.95      0.96        42
           1       0.97      0.99      0.98        72

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114
```






```
#-----------------------------------------------------------
#Cross-Validation Comparison Across All Models
#-----------------------------------------------------------
from sklearn.model_selection import cross_val_score
```

```
models = {
    "Decision Tree": dt.best_estimator_,
    "AdaBoost": ada.best_estimator_,
    "Gradient Boosting": gb.best_estimator_,
    "XGBoost": xgb_clf.best_estimator_,
    "Random Forest": rf.best_estimator_,
    "Stacked Ensemble": stack
}

cv_results = {}
for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=5, scoring="accuracy")
    cv_results[name] = scores

cv_df = pd.DataFrame(cv_results)
print("\n5-Fold CV Results:\n", cv_df)
print("\nAverage Accuracies:\n", cv_df.mean())
```

```
5-Fold CV Results:
    Decision Tree  AdaBoost  Gradient Boosting   XGBoost  Random Forest  \
0       0.929825  0.982456           0.912281  0.956140       0.938596
1       0.921053  0.964912           0.929825  0.947368       0.956140
2       0.947368  0.991228           0.982456  0.991228       0.991228
3       0.938596  0.964912           0.973684  0.982456       0.982456
4       0.946903  0.982301           0.955752  0.964602       0.964602

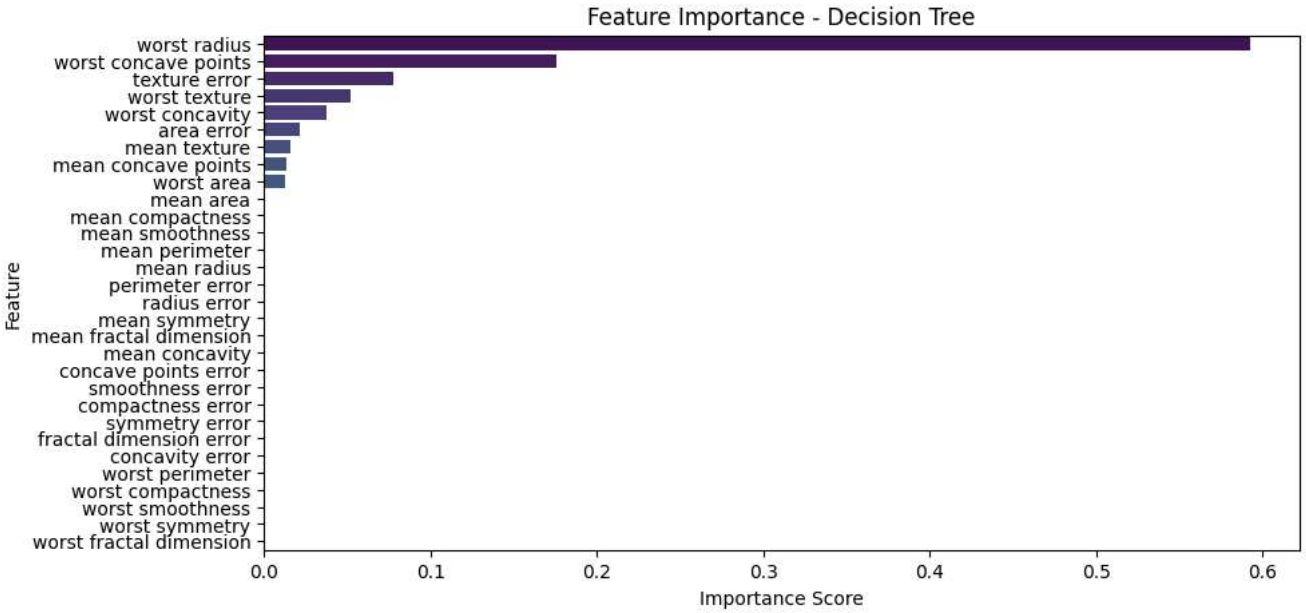
   Stacked Ensemble
0          0.947368
1          0.956140
2          0.956140
3          0.973684
4          0.973451

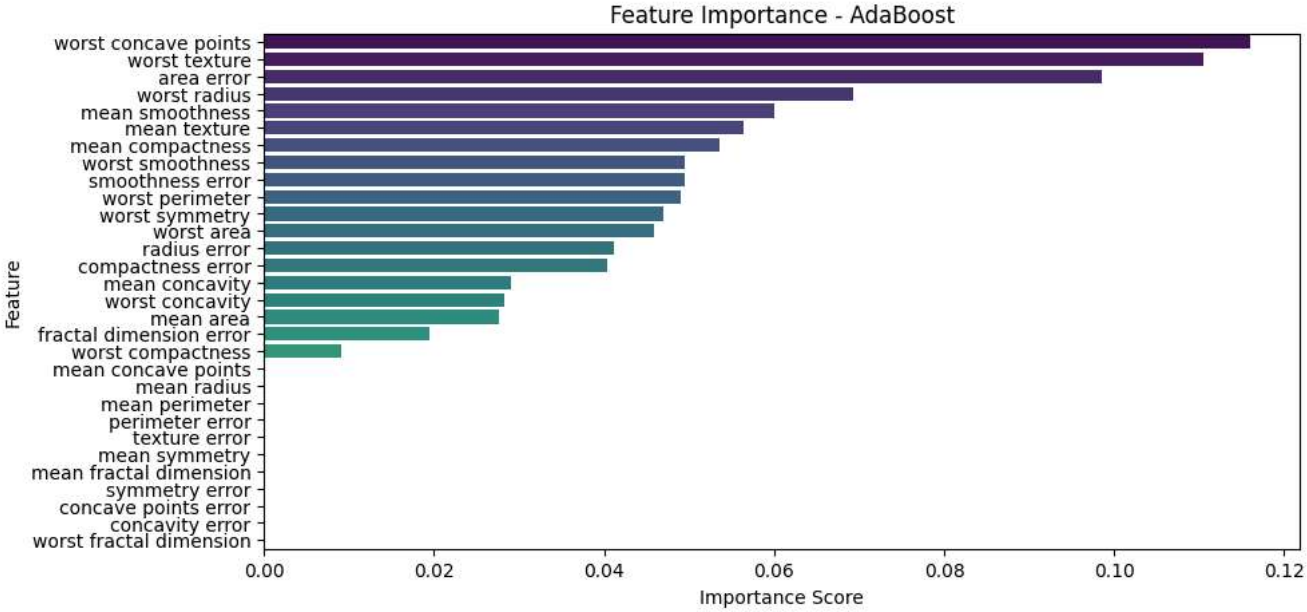
Average Accuracies:
 Decision Tree        0.936749
AdaBoost             0.977162
Gradient Boosting    0.950800
XGBoost              0.968359
Random Forest        0.966605
Stacked Ensemble     0.961357
dtype: float64
```

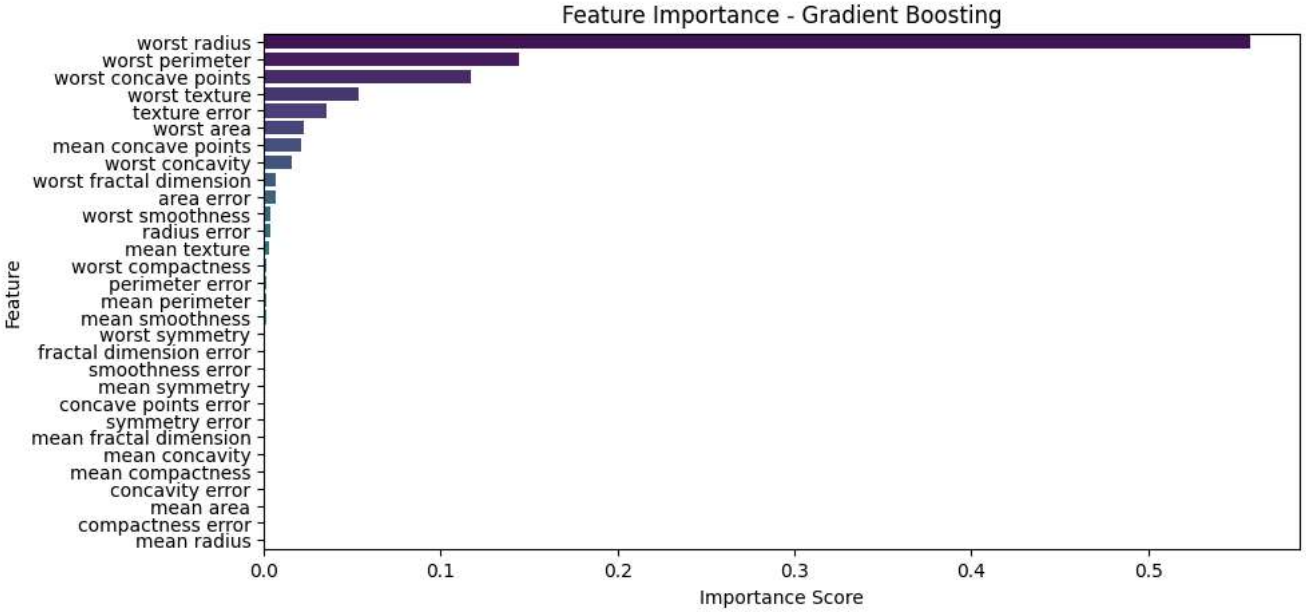
```
# ------------------------------------------------
# Feature Importance Plots
# ------------------------------------------------
def plot_feature_importance(model, feature_names, name):
    if hasattr(model, "feature_importances_"):
        importances = model.feature_importances_
        fi = pd.Series(importances, index=feature_names).sort_values(ascending=False)

        plt.figure(figsize=(10, 5))
        sns.barplot(x=fi.values, y=fi.index, palette="viridis")
        plt.title(f"Feature Importance - {name}")
        plt.xlabel("Importance Score")
        plt.ylabel("Feature")
        plt.show()

# Plot for each tree-based model
plot_feature_importance(dt.best_estimator_, data.feature_names, "Decision Tree")
plot_feature_importance(ada.best_estimator_, data.feature_names, "AdaBoost")
plot_feature_importance(gb.best_estimator_, data.feature_names, "Gradient Boosting")
plot_feature_importance(xgb_clf.best_estimator_, data.feature_names, "XGBoost")
plot_feature_importance(rf.best_estimator_, data.feature_names, "Random Forest")
```


Feature Importance - Decision Tree
worst radius
worst concave points
texture error
worst texture
worst concavity
area error
mean texture
mean concave points
worst area
mean area
mean compactness
mean smoothness
mean perimeter
mean radius
perimeter error
radius error
mean symmetry
mean fractal dimension
mean concavity
concave points error
smoothness error
compactness error
symmetry error
fractal dimension error
concavity error
worst perimeter
worst compactness
worst smoothness
worst symmetry
worst fractal dimension
Feature
0.0
0.1
0.2
0.3
0.4
0.5
0.6
Importance Score


Feature Importance - AdaBoost
worst concave points
worst texture
area error
worst radius
mean smoothness
mean texture
mean compactness
worst smoothness
smoothness error
worst perimeter
worst symmetry
worst area
radius error
compactness error
mean concavity
worst concavity
mean area
fractal dimension error
worst compactness
mean concave points
mean radius
mean perimeter
perimeter error
texture error
mean symmetry
mean fractal dimension
symmetry error
concave points error
concavity error
worst fractal dimension
Feature
0.00
0.02
0.04
0.06
0.08
0.10
0.12
Importance Score


Feature Importance - Gradient Boosting
worst radius
worst perimeter
worst concave points
worst texture
texture error
worst area
mean concave points
worst concavity
worst fractal dimension
area error
worst smoothness
radius error
mean texture
worst compactness
perimeter error
mean perimeter
mean smoothness
worst symmetry
fractal dimension error
smoothness error
mean symmetry
concave points error
symmetry error
mean fractal dimension
mean concavity
mean compactness
concavity error
mean area
compactness error
mean radius
Feature
0.0
0.1
0.2
0.3
0.4
0.5
Importance Score


Feature Importance - XGBoost
worst perimeter
worst radius